Project: Autonomous Micromouse Maze Robot

1. Introduction & Objectives

- **Project Overview:** Development of a high-speed, autonomous Micromouse robot capable of mapping and solving a standard 16x16 maze.

- **Primary Objective:** Implement Dijkstra's algorithm for optimal path planning, seamlessly integrated with a robust, closed-loop kinematic control system adhering to industrial embedded software engineering standards.

2. Hardware Architecture & Specifications

- **Microcontroller Unit (MCU):** ESP32 dual-core processor, selected for its robust RTOS capabilities and hardware floating-point unit (FPU) necessary for complex kinematics.

- **Actuators:** Two high-RPM DC motors driven by a Dual H-Bridge motor driver (e.g., TB6612FNG or L298N).

- **Sensor Suite:**

    - **Proximity:** Three I2C Time-of-Flight (ToF) laser sensors (e.g., VL53L0X) connected to the I2C1 bus. Hardware XSHUT pins are utilized for dynamic I2C address reassignment upon booting (Front: 0x29, Left: 0x30, Right: 0x31) to resolve address conflicts.

    - **Odometry:** Two absolute magnetic encoders (e.g., AS5600) connected to a dedicated high-speed I2C0 bus (400kHz) to prevent bus contention and ensure precise, jitter-free velocity measurements.

3. Software Architecture (Modular Stack)

The software is constructed using a strictly layered, Object-Oriented C (OOP in C) approach to ensure maximum portability, decoupling, and maintainability.

- **Hardware Abstraction Layer (HAL) / BSP:** Abstract interfaces (gpio.h, pwm.h, i2c_interface.h) with specific implementations for the ESP32 (bsp_esp32.cpp).

- **Device Drivers Layer:** Hardware-agnostic drivers for motors (motor.cpp), ToF sensors (laser_sensor.cpp), and encoders (encoder_sensor.cpp). Dependency injection is utilized to pass HAL interfaces into these drivers.

- **Control Layer:** Implements closed-loop kinematic control (kinematics.cpp). It features distinct PID controllers (pid_controller.cpp) for both linear velocity and angular heading, utilizing integral anti-windup mechanisms.

- **Application & Strategy Layer:** Contains the maze mapping logic, the Dijkstra path planner (dijkstra_strategy.cpp), and the central finite state machine (robot_state_machine.cpp).

4. Real-Time Operating System (RTOS) Scheduling

FreeRTOS is leveraged to distribute the computational load across the ESP32's dual cores, ensuring deterministic behavior for critical control loops while allowing background AI processing.

- **Core 1 (High Priority - Physical Control):**

    - Task_ControlLoop (500Hz): Reads encoders, computes differential odometry, executes PID algorithms, and updates motor PWM signals.

- Task_SensorUpdate (50Hz): Polls ToF laser sensors for wall detection and distance measurement.

- **Core 0 (Low Priority - AI & Logic):**

  - Task_FSMLogic (20Hz): Manages the FSM transitions and executes Dijkstra's algorithm during the planning phase without interrupting critical motor control timing.

5. Algorithmic Implementation

- **Dijkstra's Algorithm:** Optimized for embedded constraints. It utilizes static arrays instead of dynamic memory allocation (malloc) to prevent heap fragmentation. Crucially, the algorithm incorporates a 'Turn Cost' penalty to favor straight paths, thereby minimizing cumulative odometry errors caused by wheel slip during high-speed cornering.

- **Finite State Machine (FSM):** A table-driven FSM dictates the robot's macro-behavior across distinct phases: INIT (hardware setup), IDLE (wait for start signal), EXPLORE (flood-fill mapping algorithm), PLANNING (calculating optimal path via Dijkstra), and FAST_RUN (high-speed execution of the generated path).

6. Code Quality & Safety Standards (MISRA-C Inspired)

- **Memory Safety:** Strict avoidance of dynamic memory allocation post-initialization. All objects and context structures are statically allocated (static keyword).

- **Type Safety:** Mandatory use of <stdint.h> types (e.g., uint32_t, int16_t).

- **Error Handling:** All functions return standardized error codes (enums) instead of void. Null pointer checks are rigorously enforced across all APIs to prevent segmentation faults.

- **Diagnostic System:** A custom macro-based debugging system (depug.hpp) allows detailed serial logging during development. This is compiled out (zero-cost abstraction) in release mode for maximum execution speed during competitions.